Documento de Decisiones de Diseño

Sistema de gestión de citas Clínica Veterinaria

Actividad 3

Estudiante: Juan Sebastian Mier Acevedo

Materia: Programación I

FACULTAD DE CIENCIAS E INGENIERÍAS

Universidad de Manizales

Fecha: noviembre 2026

1. Introduccion

En este documento se explican las decisiones que se tomaron para crear el sistema de clinica veterinaria, desarrollado en Python utilizando Programacion Orientada a Objetos (POO).

Se presenta el problema que se va a resolver, la manera en como se organizo el programa y como se aplicaron los diferentes conceptos aprendidos sobre POO. Tambien se explica el uso de las excepciones personalizadas y las pruebas realizadas para comprobar el funcionamiento del sistema.

2. Problema que busca solucionar el sistema

En una clínica veterinaria es importante tener control de la información de las personas y sus mascotas y los veterinarios, así como de las citas médicas que se realizan.

Cuando la información no está organizada, pueden presentarse dificultades para consultar los datos de las mascotas, identificar el veterinario asignado a una cita o revisar los registros médicos de cada animal.

Para ayudar a solucionar este problema, se desarrolló un sistema en Python que permite registrar propietarios, mascotas y veterinarios, crear diferentes tipos de citas y actualizar sus estados. Además, cada mascota cuenta con un historial clínico donde se pueden guardar y consultar sus registros médicos.

3. Diseño y organización del programa

Para desarrollar el sistema de clínica veterinaria se decidió utilizar la Programación Orientada a Objetos (POO), organizando el código en diferentes clases que representan los elementos principales de una clínica veterinaria.

Se crearon las clases Propietario, Mascota y Veterinario para guardar y consultar la información correspondiente a cada uno. También se creó la clase Cita, que sirve como base para las clases CitaGeneral, CitaVacunacion y CitaControl, permitiendo manejar diferentes tipos de citas médicas.

Además, se implementaron las clases HistorialClinico y RegistroMedico para guardar y consultar los registros médicos de cada mascota. También se creó el archivo excepciones.py para definir las excepciones personalizadas utilizadas en el sistema.

Cada clase se organizó en un archivo Python independiente para mantener el código ordenado y facilitar su comprensión. Finalmente, se utilizó el archivo main.py para crear los objetos, relacionarlos y realizar las pruebas de funcionamiento del programa.

Cambios realizados durante el desarrollo

Durante el desarrollo del sistema de clínica veterinaria se realizaron varios cambios para mejorar su funcionamiento y aplicar los conceptos de Programación Orientada a Objetos. Algunos de estos cambios fueron:

- **Implementación del encapsulamiento:** se modificaron algunos atributos para controlar los cambios de sus datos. Por ejemplo, en la clase Propietario se utilizó el atributo _correo y el método cambiar_correo() para modificarlo.
- **Incorporación de validaciones:** se agregaron comprobaciones para evitar que se aceptaran datos incorrectos. Por ejemplo, en el método cambiar_edad() se estableció que los años no pueden ser negativos y que los meses deben estar entre 0 y 11.
- **Creación de excepciones personalizadas:** se implementaron excepciones para identificar mejor los errores. Por ejemplo, CorreoInvalidoError se utiliza cuando se intenta cambiar el correo de un propietario por uno vacío.
- **Sobrescritura de métodos:** se modificaron los métodos de las clases de citas para que cada una tuviera su propio comportamiento. Por ejemplo, cuando una cita de vacunación cambia a "Completada", se muestra un mensaje con el nombre de la vacuna aplicada.
- **Implementación del polimorfismo:** se organizó una lista con diferentes tipos de citas para ejecutar sus métodos mediante un mismo ciclo. Por ejemplo, en main.py se utiliza el método mostrar_informacion() para mostrar los datos de las citas generales, de vacunación y de control.
- **Incorporación del historial clínico:** se agregaron las clases HistorialClinico y RegistroMedico para guardar la información médica de las mascotas. Por ejemplo, se puede registrar una vacunación indicando su fecha, diagnóstico y tratamiento.
- **Aplicación de la composición:** se relacionaron las clases Mascota e HistorialClinico para que cada mascota tuviera su propio historial. Por ejemplo, al crear un objeto de la clase Mascota, también se crea automáticamente su historial clínico.

- **Organización de las pruebas:** se realizaron ajustes en main.py para comprobar el funcionamiento del sistema. Por ejemplo, se probaron valores incorrectos en los métodos de modificación de datos para verificar que se lanzaran las excepciones correspondientes.
- **Mejoras en la presentación de la terminal:** se organizaron los mensajes para que los resultados fueran más fáciles de leer. Por ejemplo, se separó la información mostrada durante las pruebas para identificar mejor los datos y los resultados de cada operación.

4. Aplicación de los conceptos de POO

Durante el desarrollo del sistema de clínica veterinaria se aplicaron diferentes conceptos de Programación Orientada a Objetos, como el encapsulamiento, la herencia, el polimorfismo, las clases abstractas y la composición.

Encapsulamiento

Se utilizó el encapsulamiento en las clases Propietario, Mascota y Veterinario para controlar la modificación de algunos de sus atributos. Para esto, se definieron atributos con un guion bajo (_) y métodos que permiten cambiar sus valores después de comprobar que los datos sean correctos.

Herencia

Se aplicó la herencia mediante las clases CitaGeneral, CitaVacunacion y CitaControl, que heredan de la clase Cita. Esto permitió reutilizar los atributos de las citas y evitar repetir el mismo código en cada clase.

Esta herencia representa una relación «es un» porque cada una de las clases hijas corresponde a un tipo de cita médica. Por ejemplo, una CitaVacunacion es una cita, pero tiene características propias, como el nombre de la vacuna que se va a aplicar. Lo mismo ocurre con CitaControl, que permite registrar el motivo de la revisión. De esta manera, las tres clases comparten la información general de una cita, pero cada una puede tener sus propios atributos y comportamientos.

Atributos y métodos de las clases hijas

Las clases CitaGeneral, CitaVacunacion y CitaControl heredan de la clase Cita, por lo que comparten atributos como fecha, hora, mascota, veterinario y estado. También comparten los métodos mostrar_informacion() y cambiar_estado(), aunque cada clase los implementa de acuerdo con su funcionamiento.

Cada tipo de cita tiene algunas características propias:

- **CitaGeneral:** utiliza los atributos heredados de Cita y muestra la información de una consulta general. Por ejemplo, cuando se completa, muestra el mensaje "Consulta general completada".
- **CitaVacunacion:** además de los atributos heredados, tiene el atributo vacuna, que permite identificar la vacuna correspondiente. Por ejemplo, cuando se completa, muestra un mensaje indicando qué vacuna fue aplicada.
- **CitaControl:** además de los atributos heredados, tiene el atributo motivo_control, que permite identificar el motivo de la revisión. Por ejemplo, cuando se completa, muestra un mensaje con el motivo del control realizado.

Polimorfismo

El polimorfismo se implementó mediante los métodos mostrar_informacion() y cambiar_estado(), los cuales tienen comportamientos diferentes dependiendo del tipo de cita. En el archivo main.py se utilizó una lista con diferentes tipos de citas para demostrar este concepto.

Clase abstracta

Se utilizó la clase abstracta Cita como base para las clases CitaGeneral, CitaVacunacion y CitaControl. En ella se establecieron los métodos mostrar_informacion() y cambiar_estado(), que cada tipo de cita debe realizar según su funcionamiento. Además, esta clase no se puede utilizar para crear citas directamente, sino que se deben crear a través de sus clases derivadas.

Composición

La composición se aplicó entre las clases Mascota e HistorialClinico. Cuando se crea una mascota, también se crea su historial clínico, lo que permite guardar y consultar los registros médicos correspondientes a cada animal.

Se decidió utilizar composición en lugar de herencia porque un historial clínico no es un tipo de mascota, sino que es una parte de la información que pertenece a ella. Por esta razón, la clase Mascota contiene un objeto de la clase HistorialClinico, en lugar de heredar sus atributos y métodos.

5. Excepciones personalizadas

Durante el desarrollo del sistema se implementaron excepciones personalizadas para identificar y controlar los errores que pueden presentarse al modificar algunos datos.

Para esto, se creó el archivo excepciones.py, donde se definieron tres excepciones que heredan de ValueError:

- **CorreoInvalidoError:** se utiliza cuando se intenta cambiar el correo de un propietario por uno vacío.
- **EdadInvalidaError:** se utiliza cuando se intenta asignar una edad con años negativos o meses que no estén entre 0 y 11.
- **TelefonoInvalidoError:** se utiliza cuando se intenta cambiar el teléfono de un veterinario por uno vacío.

Se decidió utilizar estas excepciones para identificar con mayor claridad los datos incorrectos y mostrar mensajes que permitan reconocer el problema. Además, en el archivo main.py se realizaron pruebas con valores inválidos para comprobar que las excepciones funcionaran correctamente.

6. Uso de inteligencia artificial

Durante el desarrollo del proyecto se utilizó ChatGPT como apoyo para resolver algunas dudas que surgieron al trabajar con Python y los conceptos de Programación Orientada a Objetos. También fue útil para revisar el código y recibir sugerencias sobre cómo mejorar la presentación de la información en la terminal.

La mayoría de las sugerencias relacionadas con la presentación de los resultados fueron aceptadas. Sin embargo, se descartaron los cambios que no eran necesarios y que podían complicar el funcionamiento del programa.

También se corrigieron algunos errores en el código. Por ejemplo, en el archivo main.py había un problema al importar la clase CitaGeneral, porque se había escrito citageneral en lugar de cita_general. Además, había un error de indentación en el bloque try/except de propietario1.cambiar_correo(), que estaba causando problemas al ejecutar el programa.

Para comprobar que las sugerencias y correcciones funcionaban, se ejecutó el programa en Visual Studio Code y se realizaron diferentes pruebas. De esta manera, se pudo verificar que los métodos funcionaran correctamente, que las excepciones detectaran los datos incorrectos y que los cambios realizados no causaran nuevos errores.

7. Referencias y herramientas consultadas

Durante el desarrollo del sistema de clínica veterinaria se utilizaron diferentes herramientas que facilitaron la programación, la organización del proyecto y la elaboración de los documentos.

- **Python:** lenguaje de programación utilizado para desarrollar el sistema y aplicar los conceptos de Programación Orientada a Objetos.
- **Visual Studio Code:** editor de código utilizado para escribir, organizar, ejecutar y probar los archivos Python del proyecto.
- **Git y GitHub:** herramientas utilizadas para llevar un control de los cambios realizados y almacenar el proyecto en un repositorio.
- **draw.io:** herramienta utilizada para elaborar el diagrama UML y representar las clases y sus relaciones.
- **ChatGPT:** se utilizó para analizar el programa, revisar si había errores y ofrecer sugerencias sobre posibles cambios para mejorar su funcionamiento y la presentación de la información en la terminal.
- **Material de estudio de la asignatura:** utilizado como referencia para aplicar los conceptos de Programación Orientada a Objetos vistos durante el curso.

8. Conclusiones

El desarrollo del sistema de clínica veterinaria permitió poner en práctica los conocimientos aprendidos sobre Programación Orientada a Objetos utilizando Python. A través de este proyecto se pudo comprender mejor cómo aplicar lo aprendido en las clases para desarrollar un programa que ayude a solucionar una necesidad real.

Durante el desarrollo se utilizaron conceptos como el encapsulamiento, la herencia, el polimorfismo, las clases abstractas y la composición. Estos conceptos fueron importantes para organizar el código, relacionar las diferentes clases y evitar repetir instrucciones innecesariamente.

También se realizaron pruebas durante el desarrollo del programa, ya que estas permiten encontrar errores y comprobar que todo funciona correctamente. Además, el uso de excepciones personalizadas ayudó a entender cómo controlar los datos incorrectos y evitar que se acepten valores que no cumplen con las condiciones establecidas.

Otra cosa importante fue aprender a organizar el proyecto en diferentes archivos y clases, lo que facilitó la comprensión del código y permitió trabajar de una manera más ordenada.

En conclusión, esta actividad permitió tener una idea más clara de cómo sería desarrollar un sistema real y conocer mejor los conceptos básicos de la Programación Orientada a Objetos. Además, ayudó a comprender cómo estos conceptos se pueden aplicar en un programa y la importancia que tienen al momento de desarrollar un sistema.